



DP-600 : Implement analytics solutions using Microsoft Fabric

Exercise 13 : Manage the semantic model lifecycle

- **Create a workspace**
- **Create a lakehouse and load data**
- **Create a semantic model**
- **Validate the semantic model with SemPy**
- **Fix the semantic model with SemPy**
- **Create a deployment pipeline**
- **Deploy content across stages**



DP-600 : Implement analytics solutions using Microsoft Fabric



Create a workspace

Create a workspace

Name *

SalesLifecycle-dev

This name is available

Description

Describe this workspace

Domain ⓘ

Assign to a domain (optional)

Learn more about workspace settings

Workspace image

Upload

Reset

Advanced ^

Contact list * ⓘ

julurikumar (Owner)

Workspace type ⓘ

What a user can do in a workspace depends on the individual user license(s) assigned to that user and the workspace type. [Learn more](#)

Power BI only workspace types

Power BI workspaces only support the use of Power BI capabilities.

Apply

Cancel

Ws_name: SalesLifecycle-dev



DP-600 : Implement analytics solutions using Microsoft Fabric



Create a workspace

Create a workspace

Name *

SalesLifecycle-prod

✔ This name is available

Description

Describe this workspace

Domain ⓘ

Assign to a domain (optional)

Learn more about workspace settings

Workspace image

Upload

Reset

Advanced ^

Contact list * ⓘ

julurikumar (Owner)

Workspace type ⓘ

What a user can do in a workspace depends on the individual user license(s) assigned to that user and the workspace type. [Learn more](#)

Power BI only workspace types

Power BI workspaces only support the use of Power BI capabilities.

Apply

Cancel

Ws_name: SalesLifecycle-prod



DP-600 : Implement analytics solutions using Microsoft Fabric



Create a lakehouse and load data

New Lakehouse

Name *

SalesLakehouse

Location *

SalesLifecycle-dev

☒ Lakehouse schemas ⓘ

Create **Cancel**

Explorer

Data items Resources Connections

+ Add data items

OneLake

SalesData

Tables

dbo

- customers
- products
- sales

Files

```
59 # Insert null CustomerKey values for data quality testing
60 sales.loc[4, "CustomerKey"] = None
61 sales.loc[14, "CustomerKey"] = None
62 sales.loc[24, "CustomerKey"] = None
63
64 # Convert CustomerKey to nullable integer to preserve type consistency
65 sales["CustomerKey"] = sales["CustomerKey"].astype(pd.Int64Dtype())
66
67 # Save as Delta tables in the lakehouse
68 spark.createDataFrame(products).write.format("delta").mode("overwrite").save()
69 spark.createDataFrame(customers).write.format("delta").mode("overwrite").save()
70 spark.createDataFrame(sales).write.format("delta").mode("overwrite").save()
71
72 print("Created tables: Products (10 rows), Customers (20 rows), Sales (200 rows)")
73 print("Data includes: 3 null CustomerKey values, 10 orphaned CustomerKey values")
```

[1] ✓ 44 sec - Session ready in 12 sec 89 ms. Command executed in 31 sec 986 ms by user1 on 4/28/2026, 9:12:39 AM

> **Spark jobs (20 of 20 succeeded)** Resources Log

Created tables: Products (10 rows), Customers (20 rows), Sales (200 rows)
Data includes: 3 null CustomerKey values, 10 orphaned CustomerKey values (99)



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ create a semantic model

New semantic model

Direct Lake semantic model name ⓘ
SalesData

Workspace ⓘ
MS Fabric Practice

Storage mode
☐ Direct Lake on OneLake ⓘ ☒ Direct Lake on SQL ⓘ

Select or deselect tables for the semantic model.

Search

☒ Select all

✓ dbo

☒ customers

☒ products

☒ sales

[Learn more about Direct Lake](#)

Confirm Cancel



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ create a semantic model

customers
City
Σ CustomerKey
CustomerName
State

products
Category
Σ ProductKey
ProductName
Σ UnitCost

sales
Σ CustomerKey
OrderDate
Σ ProductKey
Σ Quantity
Σ Sales
Σ SalesKey
Σ UnitPrice



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ Validate the semantic model with sempy

Validate the semantic model with SemPy

Before running the cells below, create a semantic model named **SalesData** from the lakehouse tables (products, customers, sales), then return to this notebook.

SemPy connects to the semantic model through the XMLA endpoint. The following cells inspect model metadata, check data quality, discover relationships, and evaluate a DAX query.



```
1 import sempy.fabric as fabric
2
3 # List all tables in the SalesData semantic model to confirm it's accessible
4 tables = fabric.list_tables("SalesData")
5 display(tables)
```

✓ 9 sec - Command executed in 9 sec 527 ms by DAG on 6/11/2026, 10:49:56 PM PySpark (Python)

> Log

Table

+ New chart

Data Wrangler

5 columns, 3 rows ▾

Table view

Download ▾

Filter by keyword

	ABC Name	ABC Description	0/1 Hidden	ABC Data Category	ABC Type	
1	customers		false		Table	
2	sales		false		Table	
3	products		false		Table	

Inspect



DP-600 : Implement analytics solutions using Microsoft Fabric

► Validate the semantic model with sempy

```
1 # List every column across all tables, showing name, data type, and parent table
2 # This helps you understand an unfamiliar model without opening Power BI Desktop
3 columns = fabric.list_columns("SalesData")
4 display(columns)
```

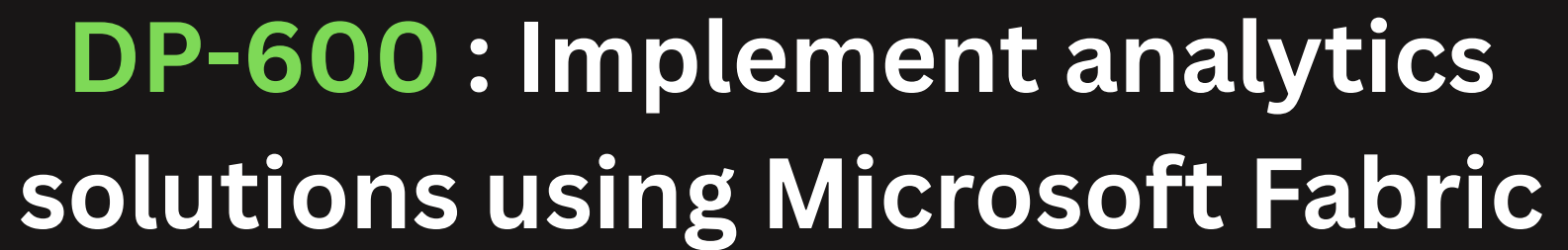
[3] ✓ <1 sec - Command executed in 915 ms by DAG on 6/11/2026, 10:51:55 PM

PySpark (Python) ▾

> Log

Table								
Table view								
ABC Table Name	ABC Column Name	ABC Description	ABC Type	ABC Data Type ...	0/1 Hidden	ABC Format String	ABC Source	
customers	CustomerKey		Data	Int64	false	0	CustomerKey	
customers	CustomerName		Data	String	false		CustomerN...	
customers	City		Data	String	false		City	
customers	State		Data	String	false		State	
sales	SalesKey		Data	Int64	false	0	SalesKey	
sales	OrderDate		Data	String	false		OrderDate	
sales	CustomerKey		Data	Int64	false	0	CustomerKey	
sales	ProductKey		Data	Int64	false	0	ProductKey	
sales	Quantity		Data	Int64	false	0	Quantity	
sales	UnitPrice		Data	Double	false		UnitPrice	
sales	Sales		Data	Double	false		Sales	
products	ProductKey		Data	Int64	false	0	ProductKey	
products	ProductName		Data	String	false		ProductName	
products	Category		Data	String	false		Category	
products	UnitCost		Data	Double	false		UnitCost	

Selected Cell 6 of 13 ce



► **Validate the semantic model with sempy**

```

1  # Evaluate a DAX query to check if the semantic model returns correct results
2  # Without relationships, the engine can't link sales to products - every
3  # category shows the same totals because the filter context doesn't propagate
4  result = fabric.evaluate_dax(
5      "SalesData",
6      """
7      EVALUATE
8      SUMMARIZECOLUMNS(
9          'products'[Category],
10         "Total Sales", SUM('sales'[Sales]),
11         "Total Quantity", SUM('sales'[Quantity])
12     )
13     ORDER BY [Total Sales] DESC
14     """
15  )
16  display(result)

```

[7] ✓ 2 sec - Command executed in 2 sec 326 ms by DAG on 6/11/2026, 10:56:52 PM

PySpark (Python) ▾

>  **Log**

Table

+ New chart

Data Wrangler

3 columns, 3 rows

Table view

Download

Filter by keyword

	ABC products[Category]	12 [Total Sales]	12L [Total Quantity]	
1	Clothing	470485.85	953	
2	Accessories	470485.85	953	
3	Bikes	470485.85	953	

Inspect

Selected Cell 10 of 13



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ Fix the semantic model with Sempy

Fix the semantic model with Sempy

The DAX results show identical totals for every category because the semantic model has no relationships defined. Without relationships, the DAX engine can't filter sales by product category.

The `connect_semantic_model` function opens a read/write connection to the semantic model's Tabular Object Model (TOM). The following cells use TOM to create the missing many-to-one relationships, then re-run the DAX query to confirm the fix.

```
1  import Microsoft.AnalysisServices.Tabular as TOM
2
3  # Open a read/write connection to the semantic model's TOM
4  with fabric.connect_semantic_model("SalesData", readonly=False) as tom:
5
6      # Create sales[ProductKey] -> products[ProductKey] (many-to-one)
7      rel_product = TOM.SingleColumnRelationship()
8      rel_product.Name = "sales_to_products"
9      rel_product.FromColumn = tom.model.Tables["sales"].Columns["ProductKey"]
10     rel_product.ToColumn = tom.model.Tables["products"].Columns["ProductKey"]
11     rel_product.FromCardinality = TOM.RelationshipEndCardinality.Many
12     rel_product.ToCardinality = TOM.RelationshipEndCardinality.One
13     tom.model.Relationships.Add(rel_product)
14
15     # Create sales[CustomerKey] -> customers[CustomerKey] (many-to-one)
16     rel_customer = TOM.SingleColumnRelationship()
17     rel_customer.Name = "sales_to_customers"
18     rel_customer.FromColumn = tom.model.Tables["sales"].Columns["CustomerKey"]
19     rel_customer.ToColumn = tom.model.Tables["customers"].Columns["CustomerKey"]
20     rel_customer.FromCardinality = TOM.RelationshipEndCardinality.Many
21     rel_customer.ToCardinality = TOM.RelationshipEndCardinality.One
22     tom.model.Relationships.Add(rel_customer)
23
24     # Changes save automatically when the context manager exits
```



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ Fix the semantic model with Sempy

```
1 # Re-run the same DAX query â€” now that relationships exist, the engine
2 # filters sales by product category and returns correct per-category totals
3 result_fixed = fabric.evaluate_dax(
4     "SalesData",
5     """
6     EVALUATE
7     SUMMARIZECOLUMNS(
8         'products'[Category],
9         "Total Sales", SUM('sales'[Sales]),
10        "Total Quantity", SUM('sales'[Quantity])
11    )
12    ORDER BY [Total Sales] DESC
13    """
14 )
15 display(result_fixed)
```

[9] ✓ 2 sec - Command executed in 2 sec 284 ms by DAG on 6/11/2026, 11:02:19 PM PySpark (Python)

> Log

Table + New chart Data Wrangler 3 columns, 3 rows

Table view Download Filter by keyword

	ABC products[Category]	12 [Total Sales]	12L [Total Quantity]
1	Bikes	438558.36	305
2	Clothing	17265.52	268
3	Accessories	14661.97	380

Selected Cell 13 of 13



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ Create a deployment pipeline



Create deployment pipeline



Home



Copilot



Deployment pipelines



Workspaces



demo dev



SalesData Deployment...



SalesData



SalesData Deployment Pipeline



New Deployment





DP-600 : Implement analytics solutions using Microsoft Fabric



Deploy content across stages



SalesData Deployment Pipeline

New Deployment pipelines

Deployment history

Manage Access

Development

MS Fabric Practice

Production

demo dev

3

Production

Deploy from

Development

Deploy 2

Flat-List

Select related

Compare

Deployment history

Filter by keyword

Filter



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ Deploy content across stages

SalesData Deployment Pipeline

New Deployment pipelines

Deployment hi

Successfully deployed content To 'Production'

Development
MS Fabric Practice

Production
demo dev
Deployed: 06/11/26, 11:17 PM
Successful deployment

Production

Deploy from

Development

Deploy

Flat-List

Select related

Compare

Deployment history

Filter by keyword

Filter

Selected stage item	Type	Compared to source	Source stage item
SalesData	Semantic model	Same as source	SalesData
SalesLakehouses	Lakehouse (Preview)	Same as source	SalesLakehouses
SalesLakehouses	SQL analytics endpoint		
	Notebook	Only in source	21b-manage-semantic-model-lifecycle



DP-600 : Implement analytics solutions using Microsoft Fabric

➤ Reference

